

**Ejercicio 03. EL VORAZ y El Señor de los Anillos**

Una manada de orcos ha desembarcado para atacar Rivendel (ciudad protegida por los humanos), Aragorn (rey de los humanos) necesita definir una estrategia de contraataque en el menor tiempo posible, por lo que decide utilizar varias técnicas y ver cual da mejores resultados. Aragorn tiene información sobre los grupos en los que se han dividido los orcos (n) y cuántos orcos hay en cada uno de los grupos. Aragorn dividirá sus tropas en (n) grupos de hombres para atacarlos. De modo que la lucha será por grupos.

Aragorn tendrá que decidir qué grupo de aliados se enfrenta a qué grupo de orcos para obtener el mayor número de victorias y debe decidirlo muy rápido. Para ello hay que tener en cuenta que una batalla será ganada si el número de aliados de un grupo es mayor o igual que el número de orcos del grupo al que se enfrentan.

Por ejemplo, suponiendo que cada grupo ataca “al grupo de enfrente”:

	Grupo 1	Grupo 2	Grupo 3	Grupo 4	Grupo 5
Ejercito de orcos	10	15	5	50	70
Ejercito de humanos	10	10	3	100	60
Resultado	Victoria	Derrota	Derrota	Victoria	Derrota

Aragorn perdería la guerra 2 a 3 por no haber optimizado sus tropas.

Légolas, un aliado elfo de Aragorn, le ha dado consejo y ha creado un sistema para ganar la batalla, un `AtaqueVoraz.java`. A pesar de que Légolas tiene más años que un bosque, Aragorn no se fía porque necesita que la solución se genere muy rápido, antes de que se muevan los orcos.

Para comprobar si Légolas tiene razón, le ha pedido a Gandalf (un mago de la informática) que le haga un programa que compruebe si la solución de Légolas es más eficiente y rápida que enfrentarse a las tropas según van viniendo, pero Gandalf le ha devuelto el programa sin documentar.

Analice y documente el proyecto de modo que Aragorn pueda entenderlo. Para ello debe:

- Analizar la estructura del programa y como funciona. Archivos, paquetes, interfaces, clases, herencia... (Documento de texto)
- Realizar una depuración del programa para comprender su funcionamiento. Punto de inicio, estructuras de datos, archivos por los que se pasa, explicación del funcionamiento y análisis de resultados (Documento de texto).
- Identificar patrones de diseño, indicando cuales y donde se usan (Documento de texto)
- Anotar en el código los comentarios que crea convenientes, utilizando anotaciones (Proyecto).
- Generar los archivos JavaDoc a partir de los comentarios realizados (JavaDoc, html).

Suba a Moodle un archivo ZIP con el proyecto comentado, la documentación y los javadocs.

Resultados de aprendizaje:

4. Optimiza código empleando las herramientas disponibles en el entorno de desarrollo.

Criterios de evaluación:

- b) Se han elaborado las pruebas asociadas a la refactorización.
- c) Se ha revisado el código fuente usando un analizador de código.
- d) Se han identificado las posibilidades de configuración de un analizador de código.
- e) Se han aplicado patrones de refactorización con las herramientas que proporciona el entorno de desarrollo.
- f) Se ha realizado el control de versiones integrado en el entorno de desarrollo.
- g) Se ha documentado el código fuente mediante comentarios.
- h) Se han utilizado herramientas del entorno de desarrollo para documentar los procesos, datos y eventos.
- i) Se han utilizado herramientas del entorno de desarrollo para documentar las clases.